

Java Collections for Beginners (Tier 3)

A practical guide to advanced collection topics. We cover thread-safe collections, specialized collections for specific cases, immutable vs unmodifiable collections, performance characteristics, and important contracts you must follow.

These topics show up in senior-level interviews and are essential for production code.

What's in Tier 3?

1. Why thread-safety matters
2. `ConcurrentHashMap` and concurrent collections
3. `CopyOnWriteArrayList`
4. `BlockingQueue` for producer-consumer patterns
5. Specialized maps (`EnumMap`, `WeakHashMap`, `IdentityHashMap`)
6. Immutable vs unmodifiable collections
7. Performance and time complexity (Big-O)
8. The `equals` and `hashCode` contract
9. Fail-fast vs fail-safe iterators
10. Building an LRU cache with `LinkedHashMap`

Let us go through them one by one.

1. Why Thread-Safety Matters

Regular collections like `ArrayList`, `HashMap`, and `HashSet` are NOT thread-safe. If multiple threads use the same collection at the same time, things break in surprising ways.

What goes wrong

```
java
```

```
List<Integer> list = new ArrayList<>();

// Thread 1
new Thread(() -> {
    for (int i = 0; i < 1000; i++) {
        list.add(i);
    }
}).start();

// Thread 2
new Thread(() -> {
    for (int i = 0; i < 1000; i++) {
        list.add(i);
    }
}).start();

// Final size might be 1500 instead of 2000
// Or you might get an ArrayIndexOutOfBoundsException
// Or the list might have null values where there should be numbers
```

The internal state can become corrupted because two threads modify it at the same time. The results are unpredictable.

Two ways to fix this

1. Use a thread-safe collection like `ConcurrentHashMap` (recommended)
2. Wrap a regular collection with `Collections.synchronizedList(...)` (older approach, slower)

We will see why the first approach is better.

2. ConcurrentHashMap : Thread-Safe Map

`ConcurrentHashMap` is a thread-safe version of `HashMap`. Multiple threads can read and write to it safely without you needing to lock anything.

Basic usage

```
java
```

```
import java.util.concurrent.ConcurrentHashMap;
import java.util.Map;

Map<String, Integer> counts = new ConcurrentHashMap<>();

// Use it just like HashMap
counts.put("apple", 1);
counts.put("banana", 2);

Integer count = counts.get("apple");
```

The API is the same as `HashMap`. The difference is what happens under the hood when multiple threads use it.

Why is it better than synchronizing a HashMap?

The old way:

```
java

Map<String, Integer> map = Collections.synchronizedMap(new HashMap<>());
```

This works, but it locks the whole map for every operation. If 10 threads want to read different keys at the same time, they all have to wait their turn.

`ConcurrentHashMap` uses smarter locking. Many threads can read at the same time. Writes only lock small parts of the map. This makes it much faster under high concurrency.

Useful concurrent methods

`ConcurrentHashMap` adds methods that are safe to use even under concurrent access.

compute : safely update a value

```
java

ConcurrentHashMap<String, Integer> counts = new ConcurrentHashMap<>();

// Safely increment a counter
counts.compute("apple", (key, oldValue) ->
    oldValue == null ? 1 : oldValue + 1
);
```

This is safe even if 100 threads call it at the same time.

computeIfAbsent : create a value if missing

```
java
```

```
Map<String, List<String>> groups = new ConcurrentHashMap<>();

groups.computeIfAbsent("fruits", key -> new ArrayList<>()).add("apple");
groups.computeIfAbsent("fruits", key -> new ArrayList<>()).add("banana");
```

The lambda only runs if the key is missing. The whole operation is atomic.

merge : combine values

```
java

ConcurrentHashMap<String, Integer> counts = new ConcurrentHashMap<>();

// Add to count, or set to 1 if missing
counts.merge("apple", 1, Integer::sum);
counts.merge("apple", 1, Integer::sum);
counts.merge("apple", 1, Integer::sum);

System.out.println(counts.get("apple")); // 3
```

This is the cleanest way to count things in a concurrent context.

Important: do NOT use null

`ConcurrentHashMap` does not allow null keys or values. This is on purpose.

```
java

ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();
map.put(null, 1); // NullPointerException
map.put("key", null); // NullPointerException
```

If you try to put null, you get a crash. This avoids ambiguous "key missing or value is null" issues.

When to use ConcurrentHashMap

- You have multiple threads accessing the same map
- You want fast concurrent reads and writes
- You need atomic operations like increment-or-add

For single-threaded code, regular `HashMap` is faster and simpler.

3. CopyOnWriteArrayList : Thread-Safe List

`CopyOnWriteArrayList` is a thread-safe list. It works in an unusual way: every time you write, it makes a copy of the entire internal array.

Basic usage

```
java

import java.util.concurrent.CopyOnWriteArrayList;
import java.util.List;

List<String> list = new CopyOnWriteArrayList<>();

list.add("Alice");
list.add("Bob");

for (String name : list) {
    System.out.println(name);
}
```

Same API as `ArrayList`.

How it works

When you call `add()` or `remove()`, the list copies its entire internal array, modifies the copy, then replaces the original. Reads always work on the latest snapshot.

This means:

- Reads are very fast (no locking)
- Writes are very slow (copy the whole array)
- Iterators see a snapshot, so no `ConcurrentModificationException`

When to use it

- Many reads, very few writes
- You want safe iteration even when other threads modify the list

A common use case is for event listeners. You add a few listeners at startup, then read the list many times to fire events.

```
java
```

```
List<EventListener> listeners = new CopyOnWriteArrayList<>();

// Adding listeners is rare
listeners.add(new MyListener());

// Firing events happens often
for (EventListener listener : listeners) {
    listener.onEvent(event);
}
```

When NOT to use it

- Frequent writes (the copying gets expensive)
- Large lists (each write copies the whole thing)

For high-write scenarios, use a different approach like `Collections.synchronizedList` with explicit synchronization.

4. BlockingQueue : Producer-Consumer Patterns

A `BlockingQueue` is a thread-safe queue with two special features:

- If you try to take from an empty queue, the thread waits until something is available
- If the queue has a max size and is full, adding waits until space frees up

This is perfect for the producer-consumer pattern.

Common implementations

- `LinkedBlockingQueue`: unbounded by default
- `ArrayBlockingQueue`: fixed-size
- `PriorityBlockingQueue`: priority-ordered
- `SynchronousQueue`: hand-off between threads

Basic usage

```
java
```

```
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;

BlockingQueue<String> queue = new LinkedBlockingQueue<>(10); // max 10 items

// Producer thread
new Thread(() -> {
    try {
        queue.put("task 1"); // waits if full
        queue.put("task 2");
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}).start();

// Consumer thread
new Thread(() -> {
    try {
        while (true) {
            String task = queue.take(); // waits if empty
            System.out.println("Processing: " + task);
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}).start();
```

`put` blocks if the queue is full. `take` blocks if the queue is empty. This naturally throttles producers and consumers.

Method choices

Action	Throws	Returns null/false	Blocks	Times out
Add to queue	<code>add(e)</code>	<code>offer(e)</code>	<code>put(e)</code>	<code>offer(e, time, unit)</code>
Take from queue	<code>remove()</code>	<code>poll()</code>	<code>take()</code>	<code>poll(time, unit)</code>
Look at front	<code>element()</code>	<code>peek()</code>	(none)	(none)

For producer-consumer, use `put` and `take`. They block, which is what you want.

Common use case: worker pool

```
java
```

```

BlockingQueue<Task> tasks = new LinkedBlockingQueue<>();

// Many worker threads
for (int i = 0; i < 5; i++) {
    new Thread(() -> {
        while (true) {
            try {
                Task task = tasks.take();
                task.execute();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                break;
            }
        }
    }).start();
}

// Main thread submits work
tasks.put(new Task("download"));
tasks.put(new Task("process"));

```

This is a clean way to build a simple thread pool.

5. Specialized Maps

There are some maps designed for specific use cases.

EnumMap : Optimized for Enum Keys

If your map keys are an enum, use `EnumMap`. It is faster and uses less memory than `HashMap`.

```

java
import java.util.EnumMap;

enum Day { MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY }

EnumMap<Day, String> schedule = new EnumMap<>(Day.class);

schedule.put(Day.MONDAY, "Team meeting");
schedule.put(Day.FRIDAY, "Code review");

schedule.forEach((day, task) ->
    System.out.println(day + ": " + task)
);

```

You must pass the enum class to the constructor.

Why is it faster?

Internally, `EnumMap` uses an array indexed by the enum's ordinal. No hashing needed. Lookups are very fast.

It also keeps keys in their natural enum order (the order they are declared).

IdentityHashMap: Compares with == Instead of equals

A regular `HashMap` uses `equals()` to compare keys. `IdentityHashMap` uses `==` instead.

```
java

import java.util.IdentityHashMap;
import java.util.HashMap;
import java.util.Map;

Map<String, Integer> regular = new HashMap<>();
Map<String, Integer> identity = new IdentityHashMap<>();

String s1 = new String("hello");
String s2 = new String("hello");

regular.put(s1, 1);
regular.put(s2, 2);
System.out.println(regular.size()); // 1 (s1.equals(s2) is true)

identity.put(s1, 1);
identity.put(s2, 2);
System.out.println(identity.size()); // 2 (s1 != s2, different objects)
```

This is rare in everyday code but useful for things like serialization, where you want to track distinct objects even if they look equal.

WeakHashMap: Keys Can Be Garbage Collected

A regular `HashMap` keeps a strong reference to its keys. As long as the map is alive, the keys cannot be garbage collected.

`WeakHashMap` holds keys as weak references. If no one else has a reference to the key, the entry can be removed automatically.

```
java
```

```
import java.util.WeakHashMap;

WeakHashMap<Object, String> map = new WeakHashMap<>();

Object key = new Object();
map.put(key, "value");

System.out.println(map.size()); // 1

key = null; // remove our reference
System.gc(); // hint to garbage collector

// Eventually, the entry will be removed automatically
```

When to use WeakHashMap

- Caches where you do not want to prevent garbage collection
- Listener registrations where you do not want to keep listeners alive
- Metadata associated with objects you do not own

It is a niche tool. Use it only when you understand the lifecycle of your keys.

6. Immutable vs Unmodifiable Collections

These two terms sound the same but mean different things. The difference matters.

Unmodifiable view

A wrapper around a mutable collection. You cannot modify through the wrapper, but the underlying collection can still change.

```
java

List<Integer> mutable = new ArrayList<>(List.of(1, 2, 3));
List<Integer> view = Collections.unmodifiableList(mutable);

view.add(4); // crash: UnsupportedOperationException

mutable.add(4); // works!
System.out.println(view); // [1, 2, 3, 4]
```

The view reflects changes to the underlying list. It just does not let you make those changes through itself.

Immutable collection

A truly immutable collection has no underlying mutable list. It cannot change at all.

```
java

List<Integer> immutable = List.of(1, 2, 3);
immutable.add(4);    // crash, same as before

// There is no underlying list to modify
```

`List.of()`, `Set.of()`, and `Map.of()` give you immutable collections.

Defensive copy

To make a truly immutable copy from a mutable list:

```
java

List<Integer> mutable = new ArrayList<>(List.of(1, 2, 3));
List<Integer> safeCopy = List.copyOf(mutable);

mutable.add(4);
System.out.println(safeCopy);    // [1, 2, 3] (unchanged)
```

`List.copyOf()` (and `Set.copyOf()`, `Map.copyOf()`) create a real copy that does not change.

Java 16+: `stream.toList()`

```
java

List<Integer> result = nums.stream()
    .filter(n -> n > 0)
    .toList();

result.add(100);    // crash, the list is unmodifiable
```

`toList()` returns an unmodifiable list. This is the modern default for stream pipelines.

Quick comparison

Type	Can the wrapper be modified?	Can the underlying change?
<code>Collections.unmodifiableList(list)</code>	No	Yes
<code>List.of(...)</code>	No	No (no underlying)
<code>List.copyOf(list)</code>	No	No (it is a copy)
<code>stream().toList()</code>	No	No

Why this matters

When you return a collection from a method, returning an unmodifiable view is NOT enough if the caller knows about your internal list.

```
java

public class Service {
    private List<String> items = new ArrayList<>();

    // BAD: caller can still modify the underlying list if they have access
    public List<String> getItemsView() {
        return Collections.unmodifiableList(items);
    }

    // GOOD: real defensive copy
    public List<String> getItemsCopy() {
        return List.copyOf(items);
    }
}
```

For APIs you do not fully control, prefer real copies.

7. Performance and Time Complexity

A common interview topic. Know the time complexity of common operations.

Big-O quick refresher

- $O(1)$: constant time, very fast
- $O(\log n)$: logarithmic, fast even for large data
- $O(n)$: linear, time grows with size
- $O(n^2)$: quadratic, slow for large data

Time complexity table

Operation	ArrayList	LinkedList	HashMap	TreeMap	HashSet	TreeSet
Add	$O(1)^*$	$O(1)$	$O(1)^*$	$O(\log n)$	$O(1)^*$	$O(\log n)$
Add at index	$O(n)$	$O(n)$	-	-	-	-
Get by index	$O(1)$	$O(n)$	-	-	-	-

Operation	ArrayList	LinkedList	HashMap	TreeMap	HashSet	TreeSet
Get by key/value	-	-	$O(1)^*$	$O(\log n)$	$O(1)^*$	$O(\log n)$
Remove by index	$O(n)$	$O(n)$	-	-	-	-
Remove by value	$O(n)$	$O(n)$	$O(1)^*$	$O(\log n)$	$O(1)^*$	$O(\log n)$
Contains	$O(n)$	$O(n)$	$O(1)^*$	$O(\log n)$	$O(1)^*$	$O(\log n)$
Iterate all	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$

The * means "average case." Worst case for HashMap operations can be $O(\log n)$ due to internal tree structure when there are many hash collisions.

Why ArrayList add is $O(1)$ "amortized"

When the array is full, ArrayList creates a new bigger array (usually 1.5x size) and copies items. This is $O(n)$. But it does not happen on every add. Spread across many adds, the average is $O(1)$. This is called "amortized constant time."

Memory overhead

Different collections have different memory costs.

Collection	Memory per entry
Array	Lowest
ArrayList	Small overhead
HashMap	Medium
LinkedHashMap	A bit more
TreeMap	Higher
LinkedList	High

For large data, this can matter. For small data, do not worry about it.

Practical guide for choosing

Use case	Best choice
Need fast access by index	ArrayList
Need fast key-value lookups	HashMap

Use case	Best choice
Need sorted keys	TreeMap
Need unique items, fast contains	HashSet
Need unique items in sorted order	TreeSet
Need a queue or stack	ArrayDeque
Need top-K or priority-based	PriorityQueue
Need thread-safe map	ConcurrentHashMap
Many readers, few writers	CopyOnWriteArrayList
Producer-consumer between threads	BlockingQueue
Map with enum keys	EnumMap

8. The equals and hashCode Contract

If you put your own classes into `HashMap`, `HashSet`, or related collections, you must follow these rules.

The contract

1. If `a.equals(b)` is true, then `a.hashCode()` must equal `b.hashCode()`.
2. If `a.hashCode()` equals `b.hashCode()`, `a.equals(b)` may or may not be true.
3. `a.equals(a)` should always be true (reflexive).
4. If `a.equals(b)` is true, then `b.equals(a)` is true (symmetric).
5. If `a.equals(b)` and `b.equals(c)`, then `a.equals(c)` (transitive).
6. Multiple calls to `a.equals(b)` should return the same result if a and b do not change.

The most important rule is #1. If you break it, hash-based collections will misbehave.

Why it matters

Hash-based collections use `hashCode()` to find a bucket, then `equals()` to find the right item in that bucket. If two equal objects have different hash codes, the collection looks in the wrong bucket and cannot find them.

Example of breaking the contract

```
java
```

```

class Point {
    int x, y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Point p)) return false;
        return x == p.x && y == p.y;
    }

    // Forgot to override hashCode!
}

Set<Point> set = new HashSet<>();
set.add(new Point(1, 2));
set.contains(new Point(1, 2));    // false! the Set cannot find it

```

Always override both methods together.

Correct implementation

```

java

class Point {
    int x, y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Point p)) return false;
        return x == p.x && y == p.y;
    }

    @Override
    public int hashCode() {
        return Objects.hash(x, y);
    }
}

```

`Objects.hash(...)` is a clean way to compute a hash code from multiple fields.

Easier: use records

If you can, use a record. It generates `equals` and `hashCode` for you correctly.

```
java
record Point(int x, int y) {}
```

That is it. You get correct `equals`, `hashCode`, and `toString` for free.

Important: do not put mutable fields in equals/hashCode

If a field used in `equals` or `hashCode` changes after putting the object into a `HashMap`, the map cannot find it anymore.

```
java
class Person {
    String name;

    @Override
    public int hashCode() {
        return name.hashCode();
    }
}

Person p = new Person();
p.name = "Alice";

Set<Person> set = new HashSet<>();
set.add(p);

p.name = "Bob";    // changes the hash code!
set.contains(p);   // false! lost in the set
```

If you store something in a hash-based collection, do not change its identity fields afterward.

9. Fail-Fast vs Fail-Safe Iterators

When you iterate a collection and someone modifies it, what happens? It depends on the iterator type.

Fail-fast iterators

Most regular collections (`ArrayList`, `HashMap`, etc.) have fail-fast iterators. They throw

`ConcurrentModificationException` if the collection is modified during iteration.

```
java

List<Integer> list = new ArrayList<>(List.of(1, 2, 3, 4));

for (Integer n : list) {
    if (n == 2) {
        list.remove(n);    // crash on next iteration
    }
}
```

This is "better fail loudly than silently behave wrong."

How to remove safely

Use the iterator's `remove` method:

```
java

Iterator<Integer> it = list.iterator();
while (it.hasNext()) {
    if (it.next() == 2) {
        it.remove();
    }
}
```

Or use `removeIf`:

```
java

list.removeIf(n -> n == 2);
```

Fail-safe iterators

Concurrent collections like `CopyOnWriteArrayList` and `ConcurrentHashMap` have fail-safe iterators. They do not throw `ConcurrentModificationException`. They iterate over a snapshot.

```
java

CopyOnWriteArrayList<Integer> list = new CopyOnWriteArrayList<>(List.of(1, 2, 3));

for (Integer n : list) {
    list.add(n + 100);    // does not crash
}
```

But the iterator may not see the changes.

Quick rule

- Regular collections: do not modify during iteration. Use the iterator's `remove` or `removeIf`.
 - Concurrent collections: safe to modify, but the iterator works on a snapshot.
-

10. Building an LRU Cache with LinkedHashMap

A "Least Recently Used" cache keeps the N most recently used items and discards the oldest. This is a classic interview question.

LinkedHashMap can do this

`LinkedHashMap` has two ordering modes: insertion order (default) and access order.

In access order mode, every time you call `get` or `put`, the entry moves to the end of the order.

The LRU implementation

```
java

import java.util.LinkedHashMap;
import java.util.Map;

public class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;

    public LRUCache(int capacity) {
        super(capacity, 0.75f, true); // true = access order
        this.capacity = capacity;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;
    }
}
```

The constructor:

- `capacity`: initial capacity
- `0.75f`: load factor (default)
- `true`: access order mode

The overridden method `removeEldestEntry` is called after every put. If it returns true, the oldest entry is removed.

Using it

```
java

LRUCache<Integer, String> cache = new LRUCache<>(3);

cache.put(1, "one");
cache.put(2, "two");
cache.put(3, "three");

cache.get(1);           // access 1, moves to end
cache.put(4, "four");   // adds 4, removes oldest (2)

System.out.println(cache);
// {3=three, 1=one, 4=four}
```

In about 15 lines, you have a working LRU cache.

Real interview question

This pattern shows up in interviews often. Practice writing it from memory.

Putting It All Together

Example 1: Thread-safe word counter

```
java

ConcurrentHashMap<String, Integer> counts = new ConcurrentHashMap<>();

ExecutorService pool = Executors.newFixedThreadPool(10);

for (String text : largeListOfTexts) {
    pool.submit(() -> {
        for (String word : text.split(" ")) {
            counts.merge(word, 1, Integer::sum);
        }
    });
}

pool.shutdown();
pool.awaitTermination(1, TimeUnit.MINUTES);

System.out.println("Total unique words: " + counts.size());
```

`merge` makes the increment atomic. No lock needed.

Example 2: Producer-consumer with bounded queue

```
java

BlockingQueue<String> queue = new ArrayBlockingQueue<>(100);

// Producer
new Thread(() -> {
    for (int i = 0; i < 1000; i++) {
        try {
            queue.put("item " + i);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            break;
        }
    }
}).start();

// Consumer
new Thread(() -> {
    while (true) {
        try {
            String item = queue.take();
            System.out.println("Got: " + item);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            break;
        }
    }
}).start();
```

If the queue is full, the producer waits. If empty, the consumer waits. Self-throttling.

Example 3: EnumMap for state machine

```
java
```



```
enum Status { PENDING, ACTIVE, PAUSED, COMPLETED, CANCELED }

EnumMap<Status, String> messages = new EnumMap<>(Status.class);
messages.put(Status.PENDING, "Waiting to start");
messages.put(Status.ACTIVE, "In progress");
messages.put(Status.PAUSED, "Paused by user");
messages.put(Status.COMPLETED, "All done");
messages.put(Status.CANCELED, "Canceled");

Status current = Status.ACTIVE;
System.out.println(messages.get(current));    // "In progress"
```

Faster than HashMap for enum keys.

Example 4: Defensive copy in API

```
java

public class TaskList {
    private final List<String> tasks = new ArrayList<>();

    public void addTask(String task) {
        tasks.add(task);
    }

    public List<String> getTasks() {
        return List.copyOf(tasks);    // defensive copy
    }
}
```

Callers cannot modify the internal list.

Example 5: Simple cache with WeakHashMap

```
java

WeakHashMap<HeavyObject, ComputedResult> cache = new WeakHashMap<>();

public ComputedResult compute(HeavyObject key) {
    return cache.computeIfAbsent(key, this::doExpensiveWork);
}
```

When the `HeavyObject` is no longer used elsewhere, the cache entry can be cleared automatically.

Common Mistakes

Mistake 1: Using HashMap from multiple threads

```
java

HashMap<String, Integer> map = new HashMap<>();
// 10 threads call map.put concurrently
```

This will eventually corrupt the map. Use `ConcurrentHashMap`.

Mistake 2: Putting null in ConcurrentHashMap

```
java

ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();
map.put(null, 1); // NullPointerException
```

Concurrent collections do not allow null. Use a placeholder or use Optional.

Mistake 3: Modifying during iteration

```
java

for (Integer n : list) {
    list.remove(n); // ConcurrentModificationException
}
```

Use `removeIf` or the iterator's `remove`.

Mistake 4: Override equals but forget hashCode

```
java

class Point {
    @Override
    public boolean equals(Object o) { /* ... */ }
    // Forgot to override hashCode!
}
```

Always override both. Or use a record.

Mistake 5: Mutable fields in hashCode

```
java
```

```
class Person {
    String name;
    @Override public int hashCode() { return name.hashCode(); }
}

set.add(person);
person.name = "new name";    // can't find it in the set anymore
```

Hash-based collections assume keys do not change.

Mistake 6: Using CopyOnWriteArrayList for write-heavy work

```
java

CopyOnWriteArrayList<Integer> list = new CopyOnWriteArrayList<>();
for (int i = 0; i < 1000000; i++) {
    list.add(i);    // each add copies the entire list, very slow
}
```

CopyOnWrite is for read-heavy workloads. Use a different approach for many writes.

Mistake 7: Confusing unmodifiable view with immutable

```
java

List<Integer> mutable = new ArrayList<>(List.of(1, 2, 3));
List<Integer> view = Collections.unmodifiableList(mutable);

mutable.add(4);
System.out.println(view);    // [1, 2, 3, 4], view changed!
```

Use `List.copyOf()` for a real immutable copy.

Mistake 8: Using LinkedHashMap as LRU without enabling access order

```
java

LinkedHashMap<Integer, String> map = new LinkedHashMap<>();
// default is insertion order, NOT access order
```

You need the 3-argument constructor with `true` for access order.

Mistake 9: Forgetting to override removeEldestEntry for LRU

```
java
```

```
LinkedHashMap<Integer, String> map = new LinkedHashMap<>(16, 0.75f, true);  
// Without removeEldestEntry, the map grows forever
```

You must override `removeEldestEntry` to actually remove old entries.

Mistake 10: Not handling InterruptedException in BlockingQueue

```
java  
  
queue.take(); // throws InterruptedException
```

Always handle this exception properly. At minimum, restore the interrupt flag.

```
java  
  
try {  
    queue.take();  
} catch (InterruptedException e) {  
    Thread.currentThread().interrupt();  
}
```

Quick Cheat Sheet

Concurrent collections

Collection	Use for
<code>ConcurrentHashMap</code>	Thread-safe map
<code>CopyOnWriteArrayList</code>	Read-heavy thread-safe list
<code>CopyOnWriteArraySet</code>	Read-heavy thread-safe set
<code>ConcurrentLinkedQueue</code>	Lock-free FIFO queue
<code>LinkedBlockingQueue</code>	Producer-consumer (unbounded)
<code>ArrayBlockingQueue</code>	Producer-consumer (bounded)
<code>PriorityBlockingQueue</code>	Thread-safe priority queue
<code>ConcurrentSkipListMap</code>	Thread-safe sorted map
<code>ConcurrentSkipListSet</code>	Thread-safe sorted set

Specialized maps

Collection	Use for
<code>EnumMap</code>	Map with enum keys
<code>IdentityHashMap</code>	Compare keys with <code>==</code> not equals
<code>WeakHashMap</code>	Allow keys to be garbage collected
<code>LinkedHashMap</code>	LRU cache, insertion order map

Immutable vs unmodifiable

Method	Type
<code>Collections.unmodifiableList(...)</code>	View, underlying can change
<code>List.of(...)</code>	True immutable
<code>List.copyOf(list)</code>	True immutable copy
<code>stream().toList()</code>	True immutable

Time complexity quick reference

Operation	ArrayList	HashMap	TreeMap
Add/Put	O(1)	O(1)	O(log n)
Get	O(1)	O(1)	O(log n)
Contains	O(n)	O(1)	O(log n)
Remove	O(n)	O(1)	O(log n)

Final Tips

- For multi-threaded code, default to `ConcurrentHashMap` over `HashMap`. The performance is great and you avoid bugs.
- `CopyOnWriteArrayList` is for read-heavy workloads. Do not use it when you write often.

3. Use `BlockingQueue` for any producer-consumer pattern. It handles all the synchronization for you.
4. For maps with enum keys, always use `EnumMap`. It is faster and clearer.
5. Returning collections from methods? Use `List.copyOf(...)` for real protection. Unmodifiable views are not enough if the caller can reach the original.
6. Always override `equals` and `hashCode` together. Or just use a record.
7. Never modify a hash-based collection's keys after putting them in. The collection cannot find them anymore.
8. Use `removeIf` or the iterator's `remove` method when removing during iteration.
9. Know the time complexity of common operations. This comes up in interviews and helps you pick the right collection.
10. Practice writing an LRU cache from memory. It is one of the most common interview questions and `LinkedHashMap` makes it surprisingly short.

You now have a deep understanding of Java collections, including the basic ones, the more specialized ones, the concurrent ones, and the contracts they rely on. This is enough to handle nearly any interview question or production problem involving Java collections.